Highway Dimension, Treewidth, & Approximation Schemes

Felix Zhou* Nathan Lo†

Abstract

Shortest path heuristics work extremely well in practice. By incurring minimal space overhead, queries can be answered in sub-linear time while examining a fraction of the entire graph. Abraham et al. introduced the notion of *highway dimension* [Abr+10] which seeks to capture properties of real world road networks. In this extended survey, we present an embedding of networks with low highway dimension into graphs of bounded treewidth due to Feldmann et al. [Fel+18]. This implies approximation algorithms for many distance based problems such as the Traveling Salesman Problem.

Contents

1 Introduction **2**

2 Definitions **2**
2.1 Metric Spaces . . . 2
2.2 Highway Dimension . . . 3
2.3 Towns and Sprawl . . . 4

3 Embedding into Graphs of Bounded Treewidth **5**
3.1 Metric Embedding . . . 6
3.2 Doubling Dimension . . . 7
3.2.1 The Split-Tree . . . 7
3.2.2 Doubling Dimension & Treewidth . . . 9
3.3 The Embedding . . . 10
3.3.1 Approximate Core Hubs . . . 10
3.3.2 Preprocessing . . . 13
3.3.3 Embed(T, j) . . . 13
3.4 Expected Distortion . . . 14
3.5 Treewidth . . . 14

4 Approximation Schemes **15**
4.1 Bounding the Aspect Ratio . . . 16
4.2 Embedding the Net . . . 17
4.3 Obtaining an Approximate Solution . . . 18
4.4 Steiner Tree & Facility Location . . . 19

5 Concluding Remarks **19**

*David R. Cheriton School of Computer Science, University of Waterloo, Waterloo, Ontario N2L 1A2, Canada. *cfzhou@uwaterloo.ca*

†David R. Cheriton School of Computer Science, University of Waterloo, Waterloo, Ontario N2L 1A2, Canada. *n5lo@uwaterloo.ca*

6 References **20**

7 Appendix **21**

1 Introduction

2 Definitions [1]

This set of notes introduces the concept of *highway dimension*, which quantifies the 'niceness' in the structure of real-life transportation networks; then provides a method to embed such 'nice' (ie. bounded highway dimension) networks into bounded treewidth graphs, while remaining faithful to the topology of the original network. As we'll briefly discuss, this embedding implies quasi-polynomial time approximation schemes for NP-complete optimization problems like the Travelling Salesman Problem and the Facility Location problem.

However first, we must make a slight detour to basic analysis.

2.1 Metric Spaces

A *metric space* generalizes the convenient properties of Euclidean space $\mathbb{R}^n$, in particular the notion of a 'nice' distance function, called a *metric*.

Definition 2.1 (Metric Space): A *metric space* (X, d) is a set X equipped with a *metric* $d : X \times X \to \mathbb{R}_{\geq 0}$ with the following properties:

- $d(x, x) = 0$ for all $x \in X$.
- $d(x, y) > 0$ whenever $x \neq y$.
- $d(x, y) = d(y, x)$ for all $x, y \in X$.
- $d(x, z) \leq d(x, y) + d(y, z)$ for all $x, y, z \in X$. This is called the *triangle inequality*.

One relevant example of a metric space is a natural embedding of an edge-weighted undirected graph $G = (V, E)$: let $\mathrm{dist}(u, v)$ denote the length of a shortest uv-path in G. We call this the *shortest path metric*. To verify that dist satisfies the properties of a metric on X, we note that the Triangle inequality holds as the shortest xz-path is always at most as long as the specific path formed by concatenating the shortest xy- and yz-paths. The rest of the properties can very easily be verified. Thus the space (X, dist) is a metric space, which we denote (K_n, dist), where $n = |V|$.

In a metric space, we denote

$$B_r(v) = \{x \in X : d(x, v) \leq r\}$$

the *closed ball of radius r about v*.

Another relevant definition will be the notion of *δ-covers* and *δ-nets* in a metric space.

Definition 2.2 (δ-Net): An δ-cover consists of a (not necessarily finite) set of points $S \subseteq X$ such that every point in X is at most distance δ from some point x_n. It is noteworthy that δ-nets in this paper will be taken to be finite, as our metric spaces will be finite. An δ-net is an δ-cover with the additional property that $d(u, v) > \delta$ for any two distinct $u, v \in S$. Note this is the same as saying the union of the $B_\delta(v)$ completely contains X, and that the additional restriction on δ-nets means no $u \neq v$ is contained in $B_\delta(v)$.

In the case where the underlying space is finite, there is a simple approach to generate δ-nets: start with an arbitrary vertex, then repeatedly take uncovered vertices until none remain. This covers every vertex

[1] Written by Nathan Lo

by the termination criterion, and the additional property of a net follows from the fact that we only add previously uncovered vertices.

2.2 Highway Dimension

Firstly, consider the following four maps of the streets near central Chicago, with increasing magnification.

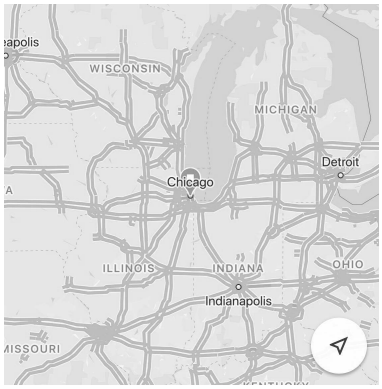

Figure 1: Interstate Highways

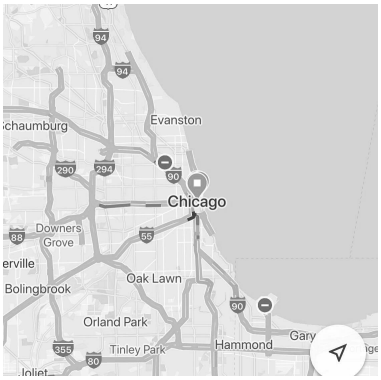

Figure 2: Local Freeways

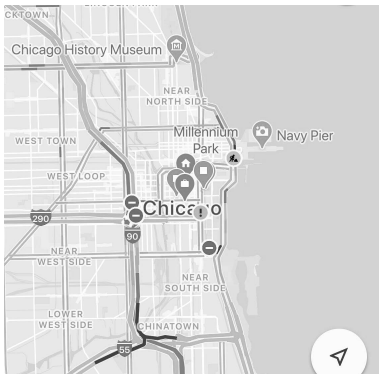

Figure 3: Approaching streets

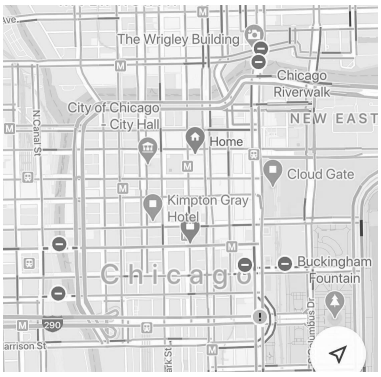

Figure 4: Street-level

Consider planning a road trip from Waterloo to Toronto. Most road guidance systems, and even most people, would suggest a trip with this general structure: start by driving from local small roads, to larger main roads, onto the local freeway, then take a long stretch on the larger highway making it most of the way to Toronto, then drive off to a local freeway in Toronto, then a larger main road in Toronto, then finally take the smaller streets in Toronto to your destination.

In reality, this is often a very good approximation of the 'true' shortest path, because transportation networks have the following nice property:

- At almost every scale, we see the emergence of 'hubs', or centres, through which lots of travellers, and thus roads, cross. In particular, in the first figure, these hubs are the major American cities, like Detroit, Indianapolis, and Chicago. In the second figure, these hubs are the smaller cities surrounding Chicago. In the third and fourth figures, these hubs are the main intersections and streets across the city core.

We will later see that the idea of 'highway dimension' builds heavily on this idea, specifically the idea that because so many roads enter and exit these hubs, most shortest paths also pass through these hubs.

One could also imagine modelling this hierarchy of scale as some tree structure, with the largest cities near the root, then smaller cities, then main streets, etc., then finding an approximate shortest path by starting at the leaf corresponding to your starting position, travelling to the closest hub, then 'zooming out', then doing the same in reverse to arrive at your destination. This idea motivates another topic: the towns-decomposition.

We now formalize the idea that any given transportation network has a nice set of hubs, which we will now call having 'bounded highway dimension':

Intuitively, a graph $G = (V, E)$ is said to have small highway dimension if for any scale $r \geq 0$ and for any vertex $v \in V$, all sufficiently long shortest paths which are close to v pass through a small set of 'hub' vertices.

Definition 2.3 (Highway Dimension): Formally, the *highway dimension* of an edge-weighted graph $G = (V, E)$ is the smallest integer k such that for some universal constant $c \geq 4$, for every $r \in \mathbb{R}_{>0}$ and every ball $B_{cr}(v)$ of radius cr, there are at most k vertices in $B_{cr}(v)$ hitting all shortest paths of length more than r that lie completely in $B_{cr}(v)$.

The closely related definition below is much nicer to work with:

Definition 2.4 (Shortest Path Cover): For a given graph G and $r \geq 0$, a *shortest path cover*, or *SPC* $\mathrm{SPC}(r) \subseteq V$ is a set of *hubs* that intersect all shortest paths of length in $(r, cr/2]$ in G. We say such a cover is *locally s-sparse for scale r* if no ball of radius $cr/2$ contains more than s vertices from $\mathrm{SPC}(r)$.

Notice that the highway dimension of a graph G is the smallest integer k such that G has a locally k-sparse shortest path cover for any scale r.

2.3 Towns and Sprawl

Continuing with the analogies to real-life, the next two definitions of *towns* and *sprawl* will also have a meaningful real-life interpretation. Intuitively, a town is a set of closely connected points which are far from everything else, and in particular far from any hubs (think: small towns are often far from major freeway intersections). The *sprawl* at a given scale is everything which is relatively close to any hub.

Now more formally,

Definition 2.5 (Towns, Sprawl): Fix a scale r, and a shortest path cover $\mathrm{SPC}(r)$. Then for any vertex $v \in V$ which is farther than $2r$ from any vertex in the SPC, we say that $T = B_r(v)$ is a *town* for scale r. The sprawl for scale r is the set of all vertices not in towns.

Now it turns out towns are relatively small, and far from everything else; and that by construction any vertex in the sprawl of scale r is close to some vertex in the shortest path cover.

Lemma 2.1 (Basic Town Properties):
Any vertex v in the sprawl is at most distance $2r$ from the closest vertex in the shortest path cover. Also, if T is a town of scale r, then $\mathrm{diam}(T) \leq r$ and $\mathrm{dist}(T, V \setminus T) > r$.

Proof (lemma 2.1): The first claim follows since any vertex v farther than $2r$ from the SPC defines a town, and thus cannot be in the sprawl.

If T is a town, generated from the vertex $v \in V$, and for the sake of contradiction, there were two vertices $u, w \in T$ with $\text{dist}(u, w) > r$, then notice $\text{dist}(u, w) \in (r, 2r] \subseteq (r, cr/2]$, so there must be some hub h along the shortest uw-path. If, say u is closer to h, then $\text{dist}(u, h) \leq r$. But then by the triangle inequality, $\text{dist}(h, v) \leq \text{dist}(h, u) + \text{dist}(v, u) \leq r + r = 2r$, contradicting the choice of v.
Again, if T is a town generated from $v \in V$, and let $u \in T$ and $w \notin T$. Then $\text{dist}(u, v) \leq r$, $\text{dist}(w, v) > r$ and $\text{dist}(v, \text{SPC}(r)) > 2r$. If we had $\text{dist}(w, u) \leq r$, then we would have

$$\text{dist}(w, v) \leq \text{dist}(w, u) + \text{dist}(u, v) \leq r + r = 2r,$$

so the length of the shortest wv-path has length in $(r, cr/2]$, so there is a hub h on this path. However this implies $\text{dist}(v, h) \leq \text{dist}(v, w) \leq 2r$, contradicting the fact that $\text{dist}(\text{SPC}(r)) > 2r$. □

Finally, we construct a laminar family of towns at specific scales, which we will use later for our embedding. Consider scales $r_i := (c/4)^i$, so that we have the property $2r_i = cr_{i-1}/2$. Thus, a ball of radius $2r_i$ around a hub of level i that covers part of the sprawl contains at most s hubs of the next lower level $i - 1$ if the shortest path covers are locally s-sparse.

We also scale the graph appropriately so that there are no shortest paths of length $(r_0, cr_0/2] = (1, c/2]$, by setting the shortest edge to be slightly more than $c/2$. Then there is no sprawl on level 0, and each vertex forms its own singleton town. The highest level we consider is $m = \lceil \log_{c/4} \text{diam}(G) \rceil$, at which $\text{SPC}(r_m) = \emptyset$ and the entire vertex set V is a town.

Lemma 2.2 (Laminar Family of Towns):
Given a graph G, the set $\mathcal{T} := \{T \subseteq V : T \text{ is a town on level } i \in N_0\}$ is a laminar family, and that any town $T \in \mathcal{T}$ on level i either has 0 or at least 2 child towns on levels below i.

Proof (lemma 2.2): Suppose two towns T_1 and T_2 cross, and that T_1 and T_2 are on levels i and j respectively, with $j \geq i$ without loss of generality. Then let $v, w \in T_1$ such that $v \in T_2$ but $w \notin T_2$. By the previous lemma, $\text{dist}(v, w) \leq \text{diam}(T_i) \leq r_i$ and $\text{dist}(v, w) \geq \text{dist}(T_2, V \setminus T_2) > r_j \geq r_i$, a contradiction.
Then we immediately get the fact that each town has either 0 or at least 2 children, because each vertex defines a singleton town on level 0, so we cannot 'miss' any vertices. Now assume a town T on level i has a child on a level $j \geq i$. Then by the previous lemma, no other town could be a child of T, since the previous lemma proves they are well-separated. This implies T has a single child, a contradiction. □

3 Embedding into Graphs of Bounded Treewidth [2]

Recall that the quadtree and kd-tree for a set of points in $\mathbb{R}^2$ are built by "splitting" the set roughly in half and recursing in the two halves. The process continues until there is only one point within our subset of the plane. See figure 5. Both data structures nicely "group" points which are close to each other since closer points are less likely to be separated by initial splits. We will generalize this concept to graphs of bounded highway dimension.

In particular, we will provide a glimpse of the following result due to Feldmann et al.

[2] Written by Felix Zhou

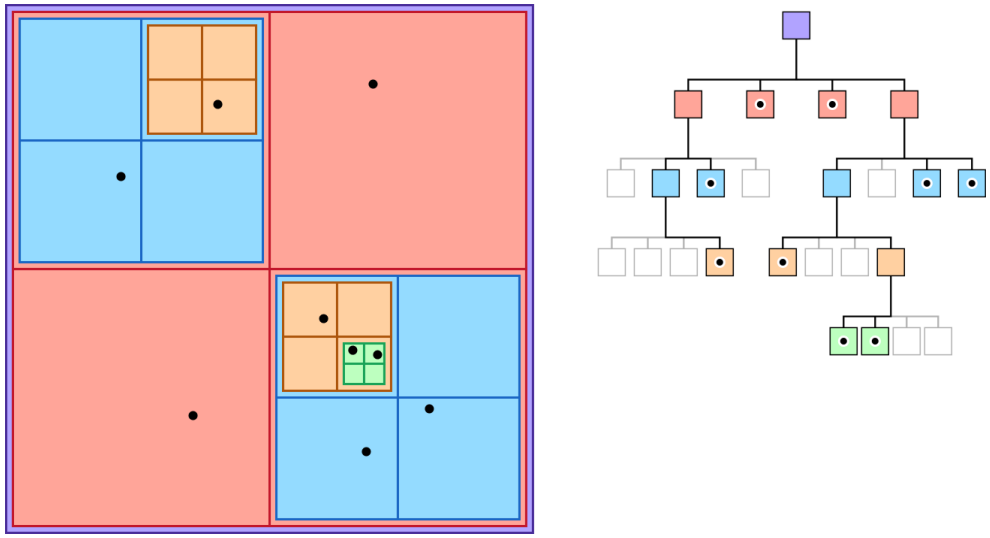

Figure 5: Points on the plane and an accompanying quadtree, courtesy of [App20].

Theorem 3.1 ([Fel+18]):
Let G be a network with highway dimension k of violation $\lambda > 0$ and aspect ratio α.
For any $\epsilon > 0$, there is a probabilistic embedding of G into the shortest path network of a $\mathcal{K}$-tree H for

$$\mathcal{K} \in (\log \alpha)^{O(\frac{1}{\lambda} \log^2(\frac{k}{\epsilon\lambda}))}$$

such that the expected distortion is at most $(1 + \epsilon)$.
Furthermore, this embedding can be computed in polynomial time.

3.1 Metric Embedding

We now give precise definitions to terms mentioned before.

Let $(X, d_X), (Y, d_Y)$ be metric spaces.

Definition 3.1 (Embedding): An *embedding* of X into Y is a map $f : X \to Y$.

For our purposes, all metric spaces are finite and an embedding never decreases the distance between two points. Thus we say that the *distortion* of an embedding $f : X \to Y$ is

$$\max_{x_1, x_2 \in X} \frac{d_Y(f(x_1), f(x_2))}{d_X(x_1, x_2)}.$$

Note that alternative definitions of distortion takes into account how much f shrinks the distance between two points as well.

Recall that a *tree decomposition* is a tree T whose vertices are subsets of V called *bags*. A valid tree decomposition "covers" all vertices and edges, while the subgraph of bags containing a given vertex $v \in V$ is a subtree. The *width* of such a tree decomposition is the maximum cardinality of a bag minus 1. Furthermore, the *treewidth* of a graph is the minimum width over all tree decompositions.

3.2 Doubling Dimension

Before we describe the construction, let us take a detour to the concept of doubling dimension. In doing so, we explore will gain more intuition of the techniques to be used as well as acquire an important building block of the embedding.

Definition 3.2 (Doubling Dimension): A metric space (X, dist) has *doubling dimension* d if for any fixed $r > 0$, it is the smallest integer such that every ball of radius r

$$B_r(v)$$

is contained in the union of at most 2^d balls of radius $\frac{r}{2}$.

We summarize the following result from Talwar.

Theorem 3.2 ([Tal04]):
Let (X, dist) be a metric space with doubling dimension d and aspect ratio α.
Fix $\epsilon > 0$, there is a probabilistic embedding of (X, dist) into the shortest path metric of a $\mathcal{K}$-tree H with treewidth at most

$$\mathcal{K} \in \left(\frac{d}{\epsilon} \log \alpha\right)^{O(d)}$$

and expected distortion $1 + \epsilon$.
Furthermore, this embedding can be computed in polynomial time.

Fix $\epsilon > 0$. Let (X, dist) be a metric space with low doubling dimension d and aspect ratio α.

3.2.1 The Split-Tree

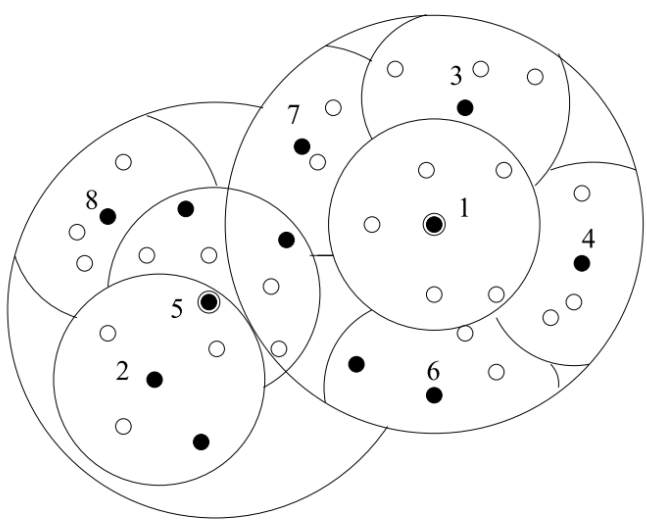

Figure 6: A metric space of low doubling dimension, courtesy of [Tal04].

Talwar's embedding employs a randomized *split-tree* decomposition of X, which acts analogously as the

quadtree for $\mathbb{R}^2$. The vertices of the split-tree are subsets of X we call *clusters*. The root corresponds to all of X and the leaves are singleton sets corresponding to the individual elements of X. Moreover, each path from some leaf to the root forms a chain with respect to $\subseteq$.

Each cluster is constructed so that it is the disjoint union of its child clusters, each of which has at most half the diameter of its parent. Intuitively, the shrinking diameter ensures that the depth of the tree is small. See figure 6 and figure 7.

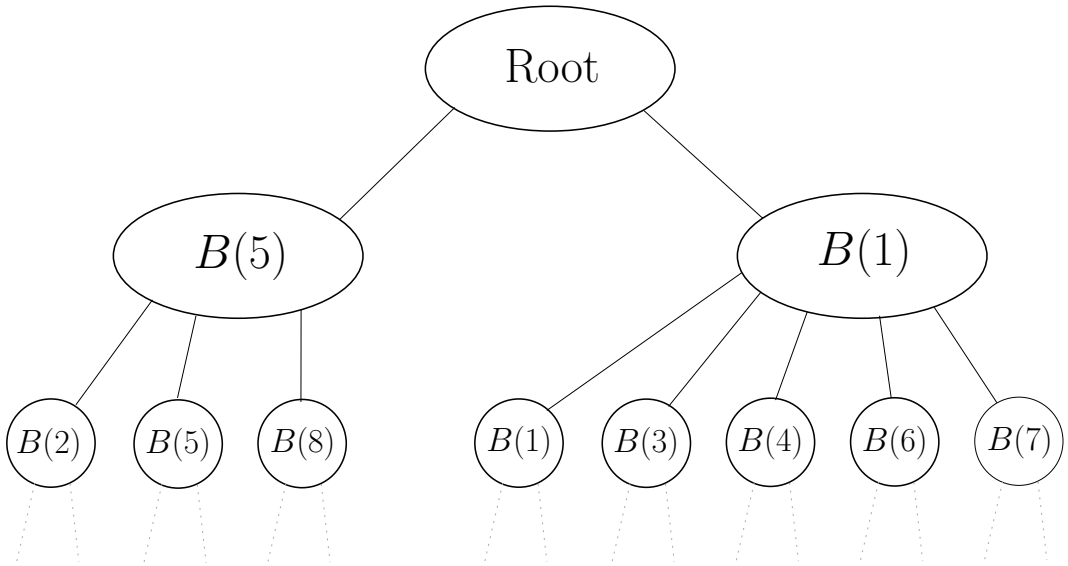

Figure 7: The accompanying split tree.

By repeatedly making a copy of some leaf cluster L and putting the copy as child of L, we can assume that all leaf clusters are at the same depth. We say the clusters of this depth are on *level* 0. Recursively, a non-leaf vertex is on level $\bar{j}$ if its children are on level $\bar{j}-1$.

Let (X, dist) be a metric space of aspect ratio α. Talwar proved the following important properties of the split-tree decomposition.

Proposition 3.3 ([Tal04]):

(i) The number of levels is $\log \alpha + 2$.

(ii) Each level $\bar{i}$ cluster has diameter at most $2^{\bar{i}+1}$.

(iii) Each cluster on level $\bar{i}$ has $2^{O(d)}$ child clusters.

Fix $\beta > 0$. Each cluster C on level $\bar{i}$ contains a greedily computable

$$\beta 2^{\bar{i}}\text{-net}$$

of C, called *portals*. The cardinality of this net is

$$\left(\frac{1}{\beta}\right)^{O(d)}.$$

3.2.2 Doubling Dimension & Treewidth

We are now ready to sketch Talwar's embedding of X. The tree decomposition $\mathscr{T}$ is computed recursively starting from the root of the split-tree. We replace the each cluster with a "fine-grained" subset.

We compute a $\beta 2^{\log_2 \alpha + 2}$-net to use as portals for the root and set the bag for the root cluster to be those portals. Recurse in each of the child clusters. Now, assume we have computed the the bag b_P for the parent P of a cluster C and C is on level $\bar{i}$. By proposition 3.3, we can compute a net of C greedily. Specifically, beginning with the portals $b_P \cap C$, compute a $\beta 2^{\bar{i}}$-net of C and set b_C to be these portals. Recurse in each of the child clusters of C.

See figure 8. The illustrated universe is some parent cluster. The dark dashed lines around the black vertices form a net for the parent and split the parent into 4 child clusters. Starting with the black vertices, we greedily add additional vertices to form smaller nets - denoted by lighter dotted lines - of each child cluster.

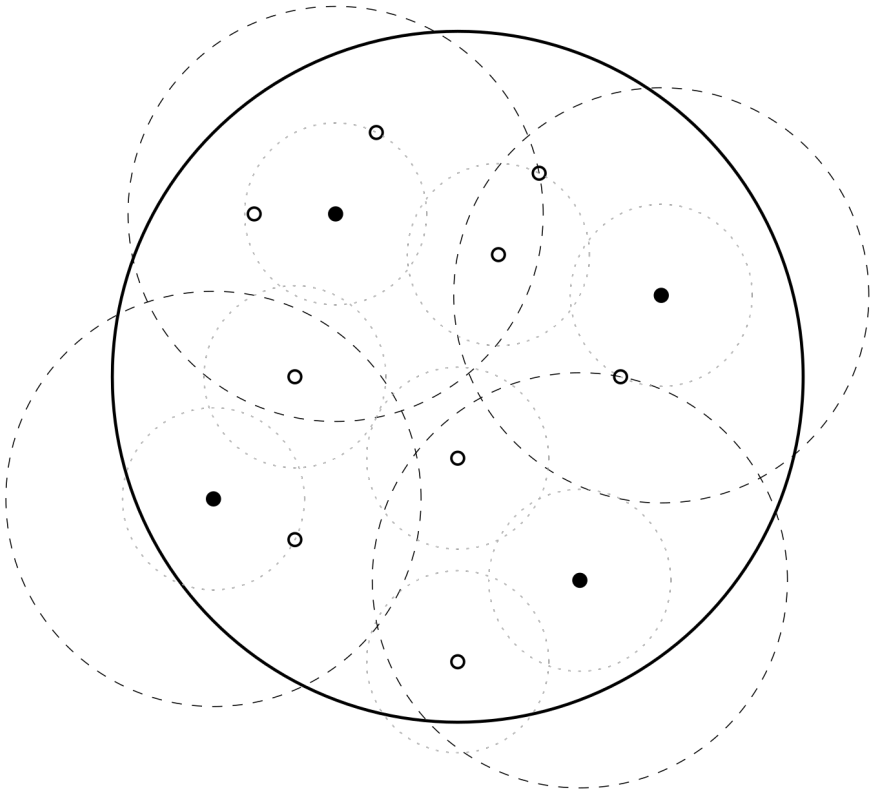

Figure 8: Obtaining a tree decomposition at a cluster of the split-tree.

Each vertex is covered since the leaves of the split-tree are the singleton sets. We do not need to worry about covering edges since we dictate which edges exist in the embedding. Finally, the connectivity condition is ensured since if a vertex is chosen to be a portal for some cluster C for the first time, it remains a portal in one of the child clusters of C. A suitable choice of β gives the treewidth bound in theorem 3.2.

The embedding H of G comes for free from the tree decomposition $\mathscr{T}$. Specifically, H has a clique on the elements of every bag b such that edges between vertices u, v contained in b has weight equal to $\text{dist}_G(u, v)$. See [Tal04] for more information. Since the detailed analysis of Talwar's embedding does not help us, we omit further details. The central idea to keep in mind is to choose small subsets of each cluster to use as an "interface" to that cluster. This concept lends itself well to graphs of low highway dimension.

3.3 The Embedding

We have already seen that towns are well-separated from each other as well as the sprawl. Similar to how portals act as as "interfaces" to their clusters, hubs are natural choices as "interfaces" between towns.

Just like how Talwar constructed his embedding using the split-tree, This embedding is constructed recursively using the towns decomposition $\mathcal{T}$. However, in contrast to a top-down recursive approach, this embedding will be computed bottom-up. Specifically, for a fixed town T, we assume that we have already computed an embedding and accompanying tree decomposition for each child town T'. Our challenge is to connect these embeddings so that distances between them are not stretched too much.

3.3.1 Approximate Core Hubs

Let us introduce some terminology and useful facts.

Definition 3.3 (Core): Let T be a town on level j and S_i the sprawl on each level $0 \leq i \leq j$. The *core* C_i of T on level i is defined recursively as follows:
$C_j := T$ and

$$C_i := S_i \cap C_{i+1}$$

for $i \leq j-1$.

Observe that $C_i \subseteq C_{i+1}$ by definition. Notice also that for any vertex $v \in C_{i+1} \setminus C_i$, v is not in the sprawl S_i, meaning it resides within a town of on level i. This means that a town on level j can be partitioned into its core C_i on level $i \leq j-1$ and its child towns on levels $\{i, i+1, \ldots, j-1\}$.

Suppose we wish to travel within vertices $v, w \in T$ belonging to child towns residing on level $i_1 \leq i_2 < i$ respectively. The above observation suggests we can take the shortest path from v to the core C_{i_1}, travel within the set of cores to reach C_{i_2}, then take the shortest path from C_{i_2} to w.

Definition 3.4 (Core Hubs): The *core hubs* of a town T on level j is the set

$$C_T := \bigcup_{i=1}^{j-1} C_i \cap \mathrm{SPC}(r_i).$$

See figure 9 for an example. The small crosses indicate the hubs of level $j-2$ and the larger ones the hubs of level $j-1$. This figure illustrates two things. First we remark that the hubs on level $j-2$ are not necessarily equal the core hubs of level $j-2$. The other point to reinforce is how the town T can be partitioned by its core hubs of level i and child towns on levels at least i.

We are now rewarded for our exploration of doubling dimension. Consider the following sketch argument.

Fix a ball $B_r(h) \cap C_T$ of radius r around a core hub h on level i. On the $i-1$-th level, there are at most s core hubs h' within $B_r(h) \cap C_T$ due to the s-sparsity of $\mathrm{SPC}(r_{i-1})$. In addition, taking balls of radius $\frac{r}{2}$ around the h''s on level $i-1$, say

$$\{B_{r/2}(h') \cap C_T : h' \in C_{i-1} \cap \mathrm{SPC}(r_{i-1})\},$$

we would cover $B_r(h) \cap C_T$. Thus the core hubs of T should have low doubling dimension.

This argument can be made rigorous and suggests that we can call upon Talwar's embedding to produce an embedding of C_T. Unfortunately, the doubling dimension of these core hubs can be shown to depend on the aspect ratio α. In particular, it would be easy to construct an example network where $\log \alpha$ grows very fast with respect to the input size.

To circumvent this issue, we introduce an important theorem of Feldmann et al.

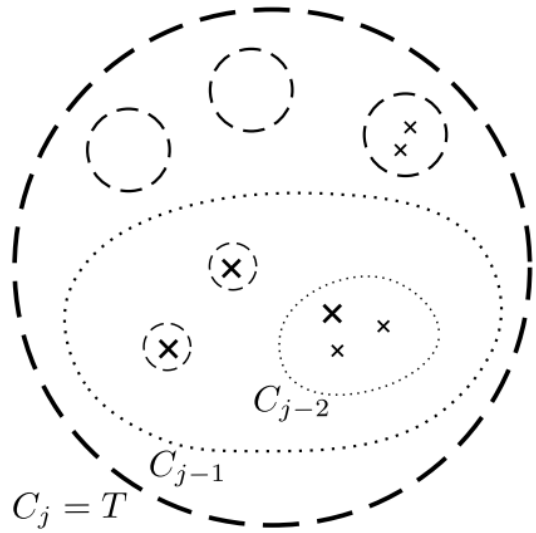

Figure 9: A town and 3 levels of cores, courtesy of [Fel+18].

Theorem 3.4 ([Fel+18]):
Let $\mathcal{T}$ be a towns decomposition of a network with highway dimension k of violation $\lambda > 0$ induced by an s-sparse shortest path cover on all levels.
Fix $\epsilon > 0$. For any town $T \in \mathcal{T}$ on level j, there exists a polynomially computable set of *approximate core hubs* $X_T \subseteq T$ such that for any core hub $h \in C_i \cap \mathrm{SPC}(r_i)$ of T on level $0 \leq i < j-1$, there is a vertex $h' \in X_T$ with

$$\mathrm{dist}_G(h, h') \leq \epsilon r_i.$$

Furthermore, the doubling dimension d of X_T satisfies

$$d \in O\left(\frac{1}{\lambda}\log\left(\frac{ks\log\left(\frac{1}{\epsilon}\right)}{\lambda}\right)\right).$$

Building on the idea of using the core as a midway point between shortest paths, we will use these approximate core hubs to "connect" the embeddings of child towns of T as mentioned at the beginning of this section to Talwar's embedding of X_T. One caveat of this construction is that we want the approximate core hubs contained in every child town T' to end up in same bag within the tree decomposition of X_T. The work around is to associate every child town T' with an approximate core hub

$$h'_T \in X_T \cap T'$$

given that $X_T \cap T' \neq \varnothing$. It can be shown that Y_T, the set of all such representatives, has doubling dimension at most $2d$. We apply Talwar's embedding to Y_T.

After computing a tree decomposition for Y_T, we can expand the bags by replacing each $h_{T'}$ with $X_T \cap T'$. Think of contracting each $X_T \cap T'$ into a super node, embedding the contraction, and "uncontracting" afterwards. See figure 10.

One final ingredient we require for the algorithm is that a bag exists every time there is a child town to connect.

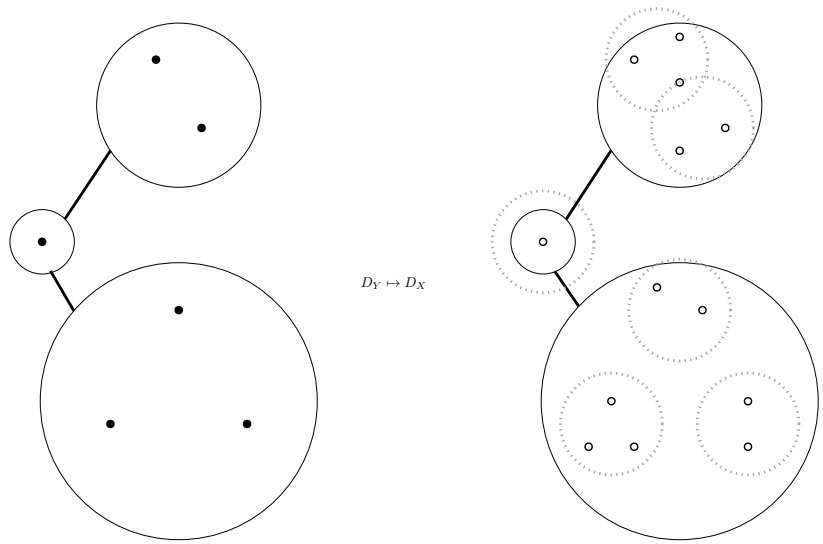

Figure 10: Expanding part of the tree decomposition of Y_T to a partial tree decomposition of X_T.

Lemma 3.5:
Let u, v be different vertices lying in different child towns of T and i such that

$$\mathrm{dist}_G(u, v) \in (r_i, r_{i+1}].$$

There is a core hub

$$h \in C_i \cap \mathrm{SPC}(r_i)$$

on level i that hits the shortest path between u and v.

Given this lemma, remark that by theorem 3.4, there is an approximate core hub close to h. This shows that there is a bag to which we can connect any child town of T.

Proof (lemma 3.5): By definition $\mathrm{SPC}(r_i)$ contains some hub h on the shortest uv-path. Recall our earlier remark that a town T can be partitioned into its core C_i from level i and child towns on level at least i. If h is not a core hub, then it is either outside of T or in a child town of T on a level at least i.
If h lies within a child town T', either $v \notin T'$ or $u \notin T'$ since they belong to different child towns. Say $v \notin T'$. By definition, h being a hub on level i prohibits it from being within a town on the same level. So T' is a town on level $i + 1$ or above. It follows that by lemma 2.1

$$\mathrm{dist}_G(v, h) > r_{i+1}.$$

But from the choice of i

$$\mathrm{dist}_G(v, h) \leq \mathrm{dist}_G(v, u) \leq r_{i+1}.$$

This is a contradiction.
Consider now the case when h lies outside of T. Let j be the level of T. By lemma 2.1,

$$\mathrm{dist}_G(v, u) \geq \mathrm{dist}_G(v, h) \geq \mathrm{dist}(T, V \setminus T) > r_j \geq r_{i+1}.$$

This is again a contradiction.
Since neither case holds h must be a core hub of level i. □

3.3.2 Preprocessing

We are ready to give the explicit embedding. In this phase of the embedding algorithm, we set up the recursive step.

1) For each level $i = 0, 1, \ldots, \left\lceil \log_{\frac{c}{4}} \operatorname{diam}(G) \right\rceil$, compute a locally $O(k \log k)$-sparse minimal shortest path cover $\mathrm{SPC}(r_i)$.
2) Let $\mathcal{T}$ be the towns decomposition based on each $\mathrm{SPC}(r_i)$.
3) Return the embedding H and corresponding D_H given by the recursive function $\texttt{Embed}\left(V, \left\lceil \log_{\frac{c}{4}} \operatorname{diam}(G) \right\rceil\right)$.

3.3.3 Embed(T, j)

Most of the steps have been explained. We summarize them here.

1) If $j = 0$, return (T, T). This handles the trivial case when we recurse to a leaf child town.
2) Compute the approximate core hubs X_T of T.
3) For levels $i = j-1, \ldots, 0$, for each child town T' of T on level i, recursively call $\texttt{Embed}(T', i)$ to produce an embedding $H_{T'}$ and accompanying tree decomposition $D_{T'}$ of T'.
4) Given one exists, choose a representative y from the set of approximate core hubs of each child town T'. Let Y_T be the set of all representatives.
5) Produce an embedding H_Y and tree decomposition D_Y of Y_T using Talwar's algorithm.
6) Expand D_Y into an embedding D_X of X_T. This also creates an embedding H_X of X_T.
7) For each child town T' of T, connect H'_T to H_T and $D_{T'}$ to D_T through a suitably chosen bag of D_X.

The remainder of this section explains the last step.

Initialize the embedding of T as $H_T := H_X$ and tree decomposition $D_T := D_X$. We will "grow" H_T and D_T throughout the "connecting" phase of the algorithm. Fix a child town T' of T, we now carefully connect $D_{T'}$ to D_T.

Choose i such that the distance from T to its closest sibling town T'' is within the interval

$$(r_i, r_{i+1}]$$

Note that the closest sibling town is not necessarily on the same level as T but exists by lemma 2.2. By lemma 3.5, there is a core hub h on level i which lies on the shortest path between T and T''. Furthermore, theorem 3.4 tells us that there is $h' \in X_T$ such that

$$\operatorname{dist}_G(h, h') \leq \epsilon r_i.$$

In particular we know that X_T is not empty and so the following is justified:

Let $h'_{T'} \in X_T$ be the closest approximate hub to T'. Choose the cluster C in the split-tree decomposition of X_T containing $h'_{T'}$ on an appropriate level. Feldmann et al. made a specific choice to suit their analysis of the expected distortion. Shortest paths from T' to core hubs close to T are preserved within expectation. At the same time, the bag b_C in D_X is still sufficiently "fine-grained" so that shortest paths to more distant hubs can be rerouted from a hub close to T' through H_X with small overhead.

We now merge the embeddings $H_T, H_{T'}$. Let b_C be the bag corresponding to C in D_T. Connect $H_{T'}$ to H_T by iterating through each pair of vertices $u \in T', v \in b_C$ and adding an edge uv with length $\operatorname{dist}_G(u, v)$ to H_T. See figure 11.

To merge the tree decompositions $D_T, D_{T'}$, connect the root of $D_{T'}$ with b_C. See figure 12. In addition, add all vertices of b_C to each bag of D'_T. This operation covers every edge of the merged H_T. Moreover, add all hubs of $X_T \cap T'$ to each bag of $D_{T'}$ as well as to b and all descendants of b in D_T originating from the bags of D_X. This ensures that the connectivity conditions still holds in the merged D_T, as we will see in section 3.5.

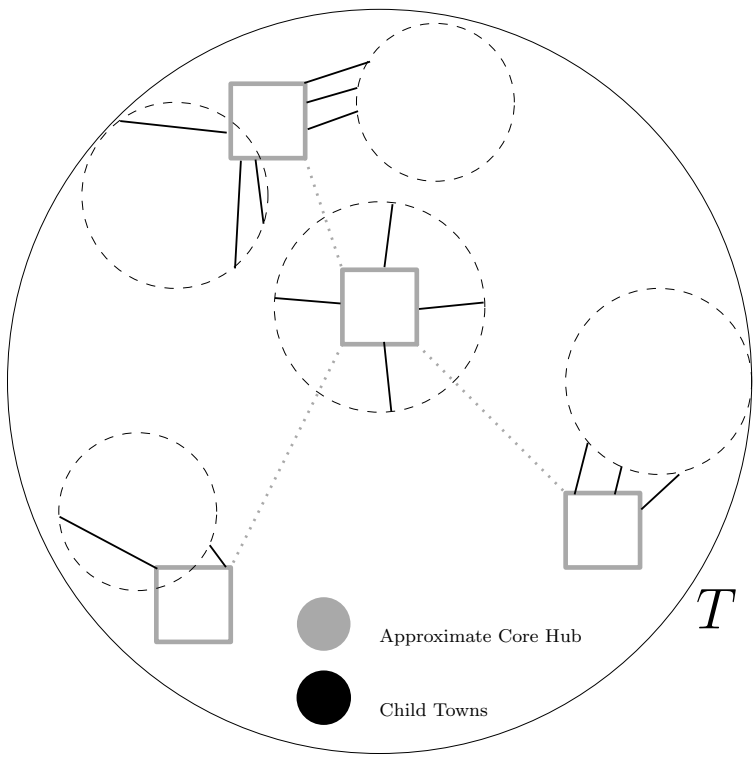

Figure 11: Merging the embeddings of child towns to H_T.

3.4 Expected Distortion

Theorem 3.6 ([Fel+18]):
The expected distortion of the embedding H of G is of order

$$1 + O(\epsilon).$$

The proof is not extremely challenging but has many many tedious steps. The essence was already explained in section 3.3.3 and relies on the specific choice of bag of D_X to which we connected $D_{T'}$. Essentially, distances between vertices in towns are directly preserved. If we wish to travel between two vertices u, v in different towns, there is a core hub h hitting the shortest path in G. In H, there is a path from u to an approximate core hub close to h, taking this path then traveling to v yields a sufficiently short path.

3.5 Treewidth [3]

In this section, we will briefly explain how we can obtain one tree-decomposition D_T of the resulting embedding H_T and argue that its tree-width is small. Recall that we obtain H_T by connecting the embeddings $H_{T'}$ of each child town of T to the embedding H_X of the approximate core hubs X_T. To construct the tree-decomposition of T, we do a similar construction: we join the tree decompositions of the child towns with D_X, and root this tree-decomposition at the highest level bag of D_X. Notice this is *almost* a valid tree-decomposition: we have yet to encode the edges connecting the child towns and their connecting bags, and the set of bags containing some vertex v may not yet be connected.

Let T' be a child town and b its connecting bag in D_X.

[3]Written by Nathan Lo

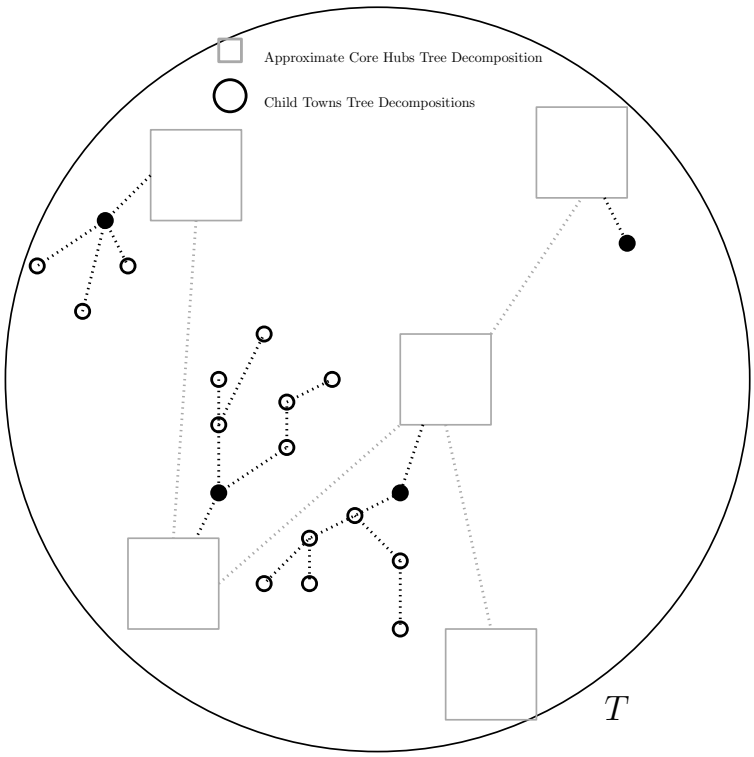

Figure 12: Merging the decompositions of child towns to D_T.

- Add all vertices of b to each bag of $D_{T'}$. This encodes the connecting edges we added to form H_T.
- Add the hubs in $X_T \cap T'$ to each bag in $D_{T'}$, to b, and all descendants of b in D_X. This ensures that the bags containing any vertex v form a connected subgraph, as needed.

The details for these claims can be found in [Fel+18].

After performing these two steps, we can now connect each child $D_{T'}$ to D_X and form a valid tree-decomposition.

Now importantly, we bound the tree-width of this resulting tree-decomposition.

Theorem 3.7 (Tree-width):
The resulting tree-width of this embedding is small, in particular at most $(\log \alpha)^{O(\log^2(\frac{ks}{\epsilon\lambda}/\lambda)}$.

Proof (theorem 3.7): Intuitive explanation of why the tree-width should be small, and a roadmap of how the proof goes, with most details omitted, citing Feldmann. □

4 Approximation Schemes [4]

We now demonstrate in detail how to use the embedding from theorem 3.1 to derive quasi-polynomial time approximation schemes (QPTAS) for the Metric Traveling Salesman Problem. Similar schemes may be obtained for the Steiner Tree and Facility Location problems.

[4] Written by Felix Zhou

Definition 4.1 (Quasi-Polynomial Time): We say an algorithm terminates in *quasi-polynomial time* if its worst case running time is order

$$2^{O(\log^c n)}$$

for some fixed constant $c > 0$.

Notice that for $c = 1$, we get a polynomial time algorithm and for $c < 1$, we get a sub-linear time algorithm. See figure 13.

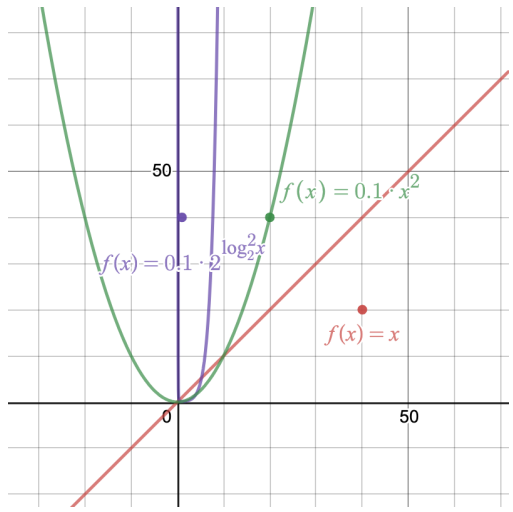

Figure 13: A comparison between a quasi-polynomial and polynomials.

Problem 1 (Metric Traveling Salesman): Given a metric space (G, dist), compute the shortest walk which starts at any vertex, visits every vertex, and returns to the starting point. We call such a walk a *tour*.

Fix $\epsilon > 0$. Let (G, dist) be an instance of the metric TSP problem where G has highway dimension k of constant violation $\lambda > 0$. The high level plan is to compute an embedding H of G using theorem 3.1 and use the fact that metric TSP is fixed parameter tractable in the treewidth. In order to guarantee quasi-polynomial running times, we need to bound the aspect ratio. We will compute a "coarse" subset of the input and embed that subset.

4.1 Bounding the Aspect Ratio

In order to bound the aspect ratio, We take a detour to recall a basic $O(1)$-approximation algorithm for metric TSP.

Proposition 4.1:
There is a polynomial time computable $O(1)$-approximation of metric TSP.

See [WS11] for details. The idea is that the shortest tour in G is at least as long as a minimum spanning tree T (in its complete graph representation). Our approximate tour essentially traverses T with some shortcuts. See figure 14.

Let OPT be the weight of an optimal tour in G. We can obtain an approximation L of OPT such that

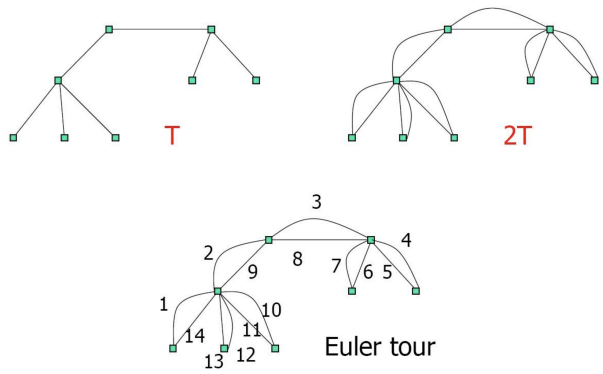

Figure 14: The MST based 2-approximation of metric TSP, courtesy of [CI11].

$L \in \Theta(\mathrm{OPT})$ by proposition 4.1. Greedily Compute a

$$\frac{L\epsilon}{n}\text{-net}$$

of (G, dist) and call it N.

By construction, the minimum distance between any two points of N is at least $\frac{L\epsilon}{n}$. To obtain an upper bound on the maximum distance, observe that a tour contains a path of length at most OPT between any two vertices. It follows that the maximum distance between any two points of N is at most L. The aspect ratio of N is then bounded above by

$$L \cdot \frac{n}{L\epsilon} \in O\left(\frac{n}{\epsilon}\right).$$

In particular, it is polynomial with respect to the input metric space (G, dist).

For $v \in V(G)$, we *assign* v to the closest $n_v \in N$. Specifically, let $\Gamma : V(G) \to N$ be given by

$$\Gamma(v) := n_v.$$

If there are any ties, break them arbitrarily. Note that if we assume the shortest paths are unique, there are no ties.

4.2 Embedding the Net

Given a good sample of G with small aspect ratio, we can compute the embedding from theorem 3.1. However, it is not clear that $G[N]$, the subgraph induced by the net is still a graph with low highway dimension. We will use $\mathcal{T}$ to generate a town decomposition of N and use that as the input to our embedding algorithm. It is important to note here that we will be solving the metric TSP on the metric space $\left(N, \mathrm{dist}\big|_N\right)$. Namely, we do not interpret N as an induced subgraph of G but a submetric of (G, dist).

For every level i and every town T on level i, create a town $\tau := T \cap N$. All properties such as laminarity, separation bounds, and diameter shown in section 2.3 are still maintained. The only additional work we need to do is on the approximate core hubs. We compute X_T as normal for each town T from which we obtained τ. The split-tree of X_T and tree decomposition D_X are also computed as before. However, we now replace every bag b of D_X with

$$\Gamma(b).$$

That is, the vertices to which the vertices of b are assigned. A re-examination of our work in section 3.5 shows that the new tree decomposition satisfies the connectivity condition and thus is a tree decomposition of some $\mathcal{K}$-tree whose vertices are $\Gamma(X_T)$. We refer to $\Gamma(X_T)$ as the *shifted approximate core hubs* [5]. For

[5] Feldmann et al. were not particularly clear on how to deal with the approximate core hubs. We made the small simplification here so the result is more obvious.

every core hub $h \in C_i \cap \mathrm{SPC}(r_i)$, there is some shifted approximate core hub of distance at most $\epsilon r_i + \frac{L\epsilon}{n}$ from h.

Consider now the embedding of N obtained using the abridged towns τ's and shifted approximate core hubs $\Gamma(X_T)$. Our analysis in section 3.4 still holds, albeit with an additional factor of $\frac{L\epsilon}{n}$. In particular, the expected shortest path length for any two vertices $u, v \in V$ within the embedding of N is of order

$$(1 + O(\epsilon)) \left(\mathrm{dist}_G(u, v) + O\left(\frac{L\epsilon}{n}\right) \right).$$

It remains to argue that the treewidth bound in theorem 3.1 still applies. Feldmann et al. achieved this by trimming the shifted approximate core hubs below an appropriate level and following the arguments from section 3.5 . We have already seen that the aspect ratio of N is of order $O\left(\frac{n}{\epsilon}\right)$. Assuming that the highway dimension k, the violation λ, and ϵ is fixed, this means the treewidth of the embedding H_N is of order

$$\log^{O(1)} n.$$

4.3 Obtaining an Approximate Solution

We now find a minimum tour in H_N using the computed tree decomposition. The dynamic programming techniques with which we are familiar from the CS762 notes [Bie20] can be applied to give a

$$O\left(\mathcal{K}^{\mathcal{K}} n\right)$$

time algorithm. Here $\mathcal{K}$ denotes the treewidth of H_N. This turns out to be quasi-polynomial time.

There is a recently developed randomized algorithm which attains a running time of

$$4^{\mathcal{K}} n^{O(1)}$$

using the so called "Cut&Count" technique [Cyg+11]. This has been improved to a deterministic algorithm at the cost of a slower run time of

$$\widetilde{O}(15^{\mathcal{K}} \mathcal{K}^{O(1)} n^2)$$

using a linear algebra based approach [Bod+13]. The $\widetilde{O}$ notation hides a polylogarithmic factor. In addition to being of innovative interest, these algorithms immediately shows us that we can solve for the optimal tour in quasi-polynomial time.

Given an exact solution S in H_N, we wish to obtain a approximate solution in G. Traverse the exact solution. For each $u \in N$, if there is an unvisited vertex $v \in V \setminus N$ which is assigned to u, then travel to v, return to u, and repeat if necessary. See figure 15. Call the tour W. Obtaining W from S incurs a cost of at most

$$2n\frac{L\epsilon}{n} = 2L\epsilon.$$

The optimal tour S in H_N has cost at most OPT since the the optimal tour in K_n contains N as a subset. Moreover, the cost function of a tour is linear in the edge costs, meaning that we may apply the linearity of expectation. This means that the weight of S within G has expected weight of order

$$(1 + O(\epsilon)) \left(\sum_{uv \in E(S)} d_G(u, v) + O\left(\frac{L\epsilon}{n}\right) \right) \subseteq (1 + O(\epsilon))(\mathrm{OPT} + O(L\epsilon)).$$

Factoring in the additional penalty incurred from modifying S to W and the fact that $L \in \Theta(\mathrm{OPT})$, we see the expected weight of W is of order

$$(1 + O(\epsilon))(1 + O(\epsilon))\,\mathrm{OPT} + O(\epsilon)\,\mathrm{OPT}\,.$$

This yields the approximation guarantee.

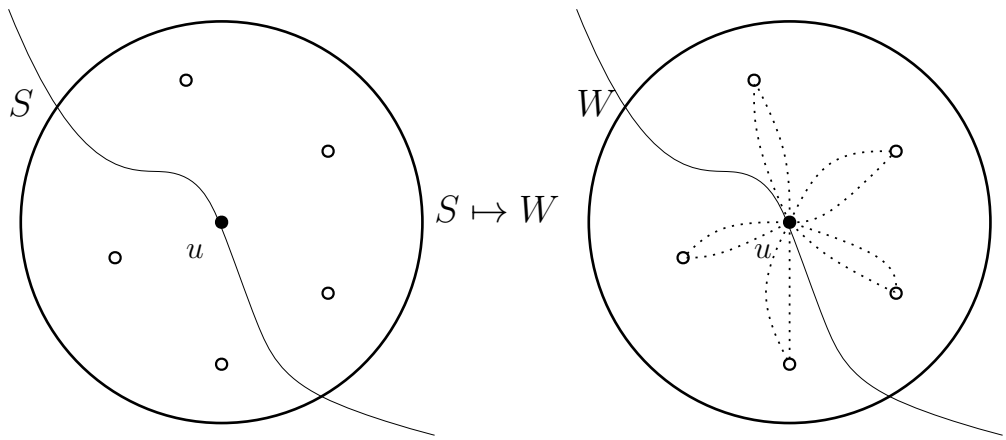

Figure 15: Obtaining W from S at a single vertex $u \in N$.

Theorem 4.2:
Let (G, dist) be a graph with highway dimension k of violation $\lambda > 0$.
There is a probabilistic QPTAS for metric TSP in G.

4.4 Steiner Tree & Facility Location

Problem 2 (Steiner Tree): Given an edge weighted graph G and a set of *terminals* $R \subseteq V$, compute a minimum cost tree in G spanning all the terminals.

Problem 3 (Facility Location): Let G be a a vertex and edge weighted graph G. We refer to a non-empty subset $W \subseteq V$ as *facilities*. The opening cost of a facility is given by the vertex weights. The connection cost of each $v \in V$ of the distance between v and the closest facility.
Find a set of facilities minimizing the opening and connection costs.

For each of the two above problems, a QPTAS may be obtained by adapting the steps of the one given for metric TSP. There are one of two minor hiccups but no new ideas, thus we defer the reader to [Fel+18].

5 Concluding Remarks [6]

All in all, we introduced the concept of highway dimension graphs and showed that we can find embeddings into graphs of bounded treewidth with arbitrarily small expected distortion due to [Fel+18]. This implies QPTAS for various distance related problems such as metric TSP and Facility Location. We did not comment on the complexity of these problems for graphs of bounded highway dimension as it has been unknown until of recent.

In [Dis+19], the authors demonstrated that metric TSP is NP-hard on graphs of highway dimension 6. They also gave a fully polynomial-time approximation scheme (FPTAS) for metric TSP, albeit only for graphs of highway dimension 1. This result was obtained by directly analyzing the treewidth of these graphs.

[6] Written by Felix Zhou

In [FS20], the authors show that Facility Location is NP-hard even for graphs of highway dimension 1. Moreover, the authors derived a PTAS for Facility Location. This result leverages the theory of metric embeddings and improves upon the work in [Fel+18].

One point of contention is the definition of highway dimension itself. The definition we presented was an early definition due to Abraham et al. [Abr+10] and was modified slightly through the violation parameter $\lambda \geq 0$ to suit the embedding by Feldmann et al. In a follow-up paper [Abr+11], the former provided a more general definition which is suited to continuous settings. In yet another update [Abr+13], Abraham et al. gave a more restrictive definition for which constant highway dimension with violation $\lambda = 0$ implies constant doubling dimension. In this third definition, Talwar's embedding could be directly applied.

The origin paper [Abr+10] also provided a way to generate graphs of low highway dimension which model transportation networks in practice. We conclude this survey by asking an open question [7]: which definition of highway dimension is most suitable to model real life transportation networks?

6 References

[Abr+10] Ittai Abraham et al. "Highway dimension, shortest paths, and provably efficient algorithms". In: *Proceedings of the Annual ACM-SIAM Symposium on Discrete Algorithms*. 2010, pp. 782–793. ISBN: 9780898717013. DOI: 10.1137/1.9781611973075.64. URL: https://www.researchgate.net/publication/220780045.

[Abr+11] Ittai Abraham et al. *VC-dimension and shortest path algorithms*. Tech. rep. PART 1. 2011, pp. 690–699. DOI: 10.1007/978-3-642-22006-7_58.

[Abr+13] Ittai Abraham et al. *Highway Dimension and Provably Efficient Shortest Path Algorithms*. Tech. rep. MSR-TR-2013-91. 2013. URL: https://www.microsoft.com/en-us/research/publication/highway-dimension-and-provably-efficient-shortest-path-algorithms/.

[App20] Apple. *Spatial and logical arrangement of an example quadtree*. 2020. URL: https://developer.apple.com/documentation/gameplaykit/gkquadtree.

[Bie20] Therese Biedl. *CS 762: Graph-theoretic algorithms Lecture notes of a graduate course University of Waterloo*. 2020.

[Bod+13] Hans L. Bodlaender et al. "Deterministic single exponential time algorithms for connectivity problems parameterized by treewidth". In: *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)* 7965 LNCS.PART 1 (2013), pp. 196–207. ISSN: 03029743. DOI: 10.1007/978-3-642-39206-1_17.

[CI11] Chandra Chekuri and Sungjin Im. *CS 598CSC: Approximation Algorithms*. 2011.

[Cyg+11] Marek Cygan et al. "Solving connectivity problems parameterized by treewidth in single exponential time". In: *Proceedings - Annual IEEE Symposium on Foundations of Computer Science, FOCS*. 2011, pp. 150–159. ISBN: 9780769545714. DOI: 10.1109/FOCS.2011.23. arXiv: 1103.0534.

[Dis+19] Yann Disser et al. "Travelling on Graphs with Small Highway Dimension". In: *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*. Vol. 11789 LNCS. 2019, pp. 175–189. ISBN: 9783030307851. DOI: 10.1007/978-3-030-30786-8_14. arXiv: 1902.07040.

[Fel+18] Andreas Emil Feldmann et al. "A $(1 + \epsilon)$-embedding of low highway dimension graphs into bounded treewidth graphs". In: *SIAM Journal on Computing* 47.4 (2018), pp. 1667–1704. ISSN: 10957111. DOI: 10.1137/16M1067196. arXiv: 1502.04588v6.

[FS20] Andreas Emil Feldmann and David Saulpic. "Polynomial Time Approximation Schemes for Clustering in Low Highway Dimension Graphs". In: (2020). arXiv: 2006.12897. URL: http://arxiv.org/abs/2006.12897.

[7] We considered ways to attack this problem but did not make much progress due to time constraints.

[Tal04] Kunal Talwar. "Bypassing the embedding". In: *Proceedings of the thirty-sixth annual ACM symposium on Theory of computing - STOC '04*. New York, New York, USA: Association for Computing Machinery (ACM), 2004, p. 281. DOI: 10.1145/1007352.1007399. URL: http://portal.acm.org/citation.cfm?doid=1007352.1007399.

[WS11] David P. Williamson and David B. Shmoys. *The Design of approximation algorithms*. Cambridge University Press, 2011.

7 Appendix

List of Figures

1 Interstate Highways . . . 3
2 Local Freeways . . . 3
3 Approaching streets . . . 3
4 Street-level . . . 3
5 Points on the plane and an accompanying quadtree, courtesy of [App20]. . . . 6
6 A metric space of low doubling dimension, courtesy of [Tal04]. . . . 7
7 The accompanying split tree. . . . 8
8 Obtaining a tree decomposition at a cluster of the split-tree. . . . 9
9 A town and 3 levels of cores, courtesy of [Fel+18]. . . . 11
10 Expanding part of the tree decomposition of Y_T to a partial tree decomposition of X_T. . . . 12
11 Merging the embeddings of child towns to H_T. . . . 14
12 Merging the decompositions of child towns to D_T. . . . 15
13 A comparison between a quasi-polynomial and polynomials. . . . 16
14 The MST based 2-approximation of metric TSP, courtesy of [CI11]. . . . 17
15 Obtaining W from S at a single vertex $u \in N$. . . . 19